# DATA 1030 FINAL REPORT

By

YANLE LYU

BROWN UNIVERSITY, 2025

DATA SCIENCE INSTITUTE

DATA 1030 2025 FALL

Github Repository: https://github.com/LYL55555/data1030-project

PROVIDENCE, RHODE ISLAND

DECEMBER 2025

# INTRODUCTION

The goal of this project is to answer a simple research question: can indoor occupancy be predicted accurately using only low-cost environmental sensors and recent time history? This question matters because many building-automation systems rely on occupancy to control ventilation and lighting, and accurate prediction can reduce energy waste while keeping comfort unchanged.

The dataset is the UCI Occupancy Detection dataset[1]. It contains time-stamped measurements of temperature, humidity, light, $CO_2$, and humidity ratio, recorded in a single office room, together with a binary occupancy label. The data show clear temporal patterns and smooth sensor dynamics, so all evaluation must respect time order. No missing values are present. Prior works show testing F-1 scores ranging 0.9-0.97, which is similar to what I got.

The aim is to build a clean time-series pipeline, compare linear and non-linear models, measure uncertainty from splitting and randomness, and interpret which features and lags drive occupancy prediction.

[1] http://archive.ics.uci.edu/dataset/357/occupancy+detection

# EDA

The dataset contains 17,893 time-stamped observations in office environment. Occupancy is imbalanced (78.9% empty vs. 21.1% occupied)(Figure 1).

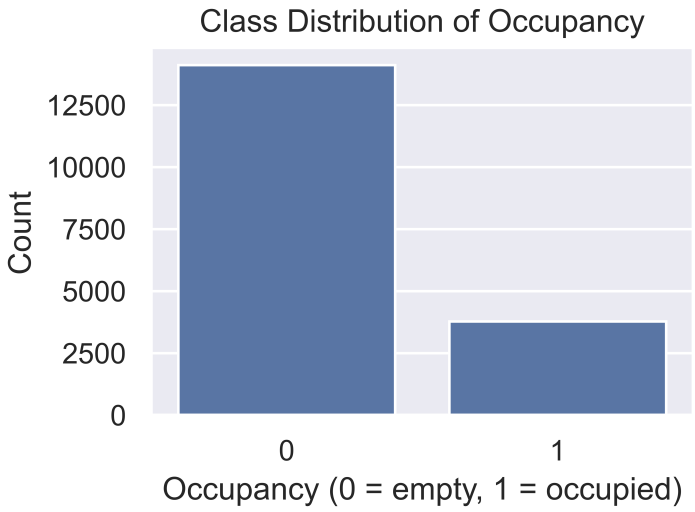


Figure 1: Class distribution of the Occupancy label.

Occupancy shows a clear daily and weekly pattern (Figure 2), indicating strong temporal dependence and non-iid behavior.

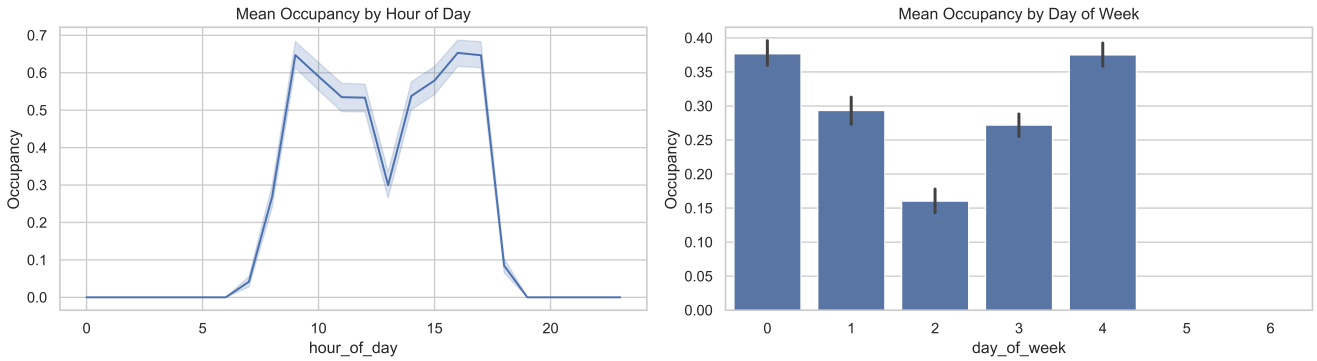


Figure 2: Mean occupancy by hour of day (left) and day of week (right).

Light is the strongest single predictor (Figure 3). Empty periods have near-zero light, while occupied periods form a distinct high-light cluster with minimal overlap.

Combining Light and $CO_2$ yields even clearer class separation (Figure 4), with occupied intervals showing high values on both axes.

Short-term temporal dependence is also strong (Figure 5): sensor autocorrelations exceed 0.95 within 10 minutes and remain high at 30 minutes. Occupancy shows the same pattern, confirming that recent history is highly informative. All of the above proves the necessity of lag features and time-aware modeling.

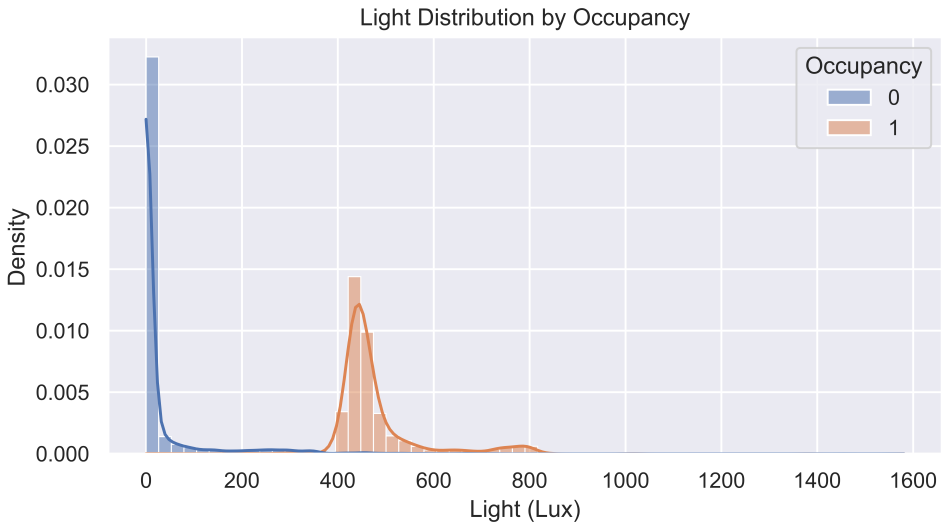


Figure 3: Light distributions for empty vs. occupied periods.

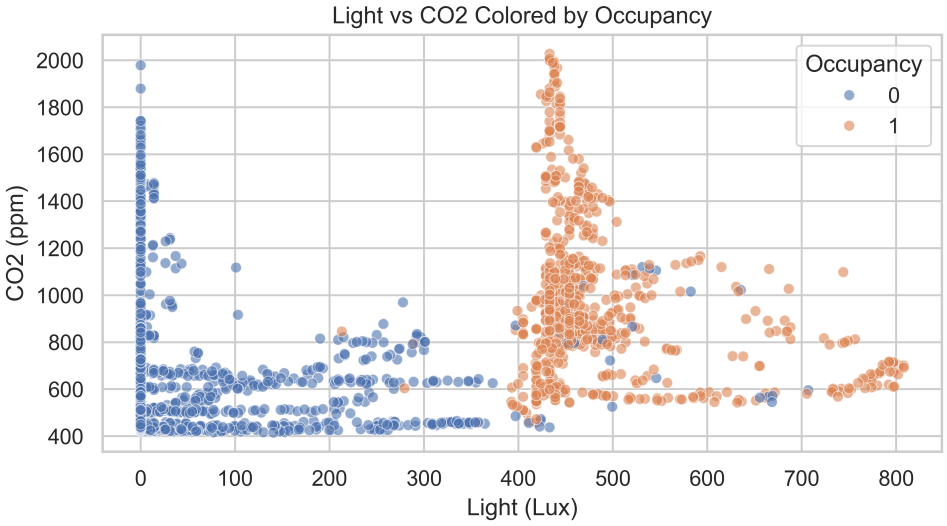


Figure 4: Light vs. $CO_2$

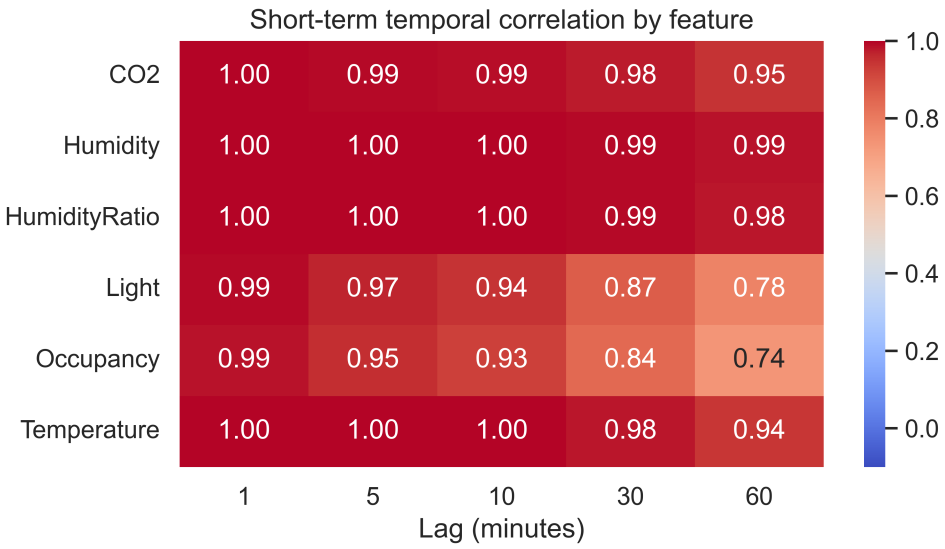


Figure 5: Short-term temporal correlation at different lag lengths

# Methods

## Feature construction

To capture the clear temporal patterns, I add two features: **hour_of_day** (0–23) and a one-hot encoded **day_of_week** (0–6). These encode the strong daily and weekly cycles in occupancy.

I created **30-minute lag features** for all sensor variables. Because the two raw files contain a gap, lags are computed separately within each file, and rows lacking a valid 30-minute history are removed.

A 30-minute lag is chosen as a compromise: very short lags (1–5 minutes) are almost identical to the current reading, while longer lags (60 minutes or 1 day) lose predictive strength. Thirty minutes remains highly correlated with the present while still capturing transitions in Light and $CO_2$.

The final feature set used in the model includes:

- Temperature, Humidity, Light, $CO_2$, HumidityRatio
- Temperature_lag30, Humidity_lag30, Light_lag30, CO2_lag30, HumidityRatio_lag30
- hour_of_day, dow_0, dow_1, dow_2, dow_3, dow_4, dow_5, dow_6

## Data splitting and cross-validation

All splits respect temporal order (60% train, 20% validation, 20% test). **TimeSeriesSplit** is applied on the training set only to assess model stability across different historical window sizes. Hyperparameters are selected using the validation set based on F1 score. The chosen parameters are fixed and final performance is reported once on the held-out test set.

## Preprocessing and ML pipeline

All sensor variables are continuous and treated as numerical features. **hour_of_day** is kept as an integer to reflect its smooth cyclic structure. **day_of_week** is one-hot encoded (`dow_0`–`dow_6`) to avoid imposing an artificial ordering. The pipeline consists of standardizing all numerical inputs followed by the model.

## Evaluation metric

Because occupancy is imbalanced (79% empty, 21% occupied), I use the **F1 score** as the main metric because it balances precision and recall and penalizes models that simply predict the majority class.

## Models and hyperparameter grids

I fit the following models:

| **Model** | **Hyperparameters tried** |
|---|---|
| DummyClassifier | Stratified Random Guesser. |
| Logistic Regression (L1) | $C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3\}$ |
| Logistic Regression (L2) | $C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3\}$ |
| Elastic Net Logistic Regression | $C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3\}$<br>$l1\_ratio \in \{0.1, 0.5, 0.9\}$ |
| KNN | $n_{neighbors} \in \{1, 3, 5, 7, 9, 11, 15, 20, 25, 30, 40, 50, 100\}$<br>`weights` $\in$ {`uniform`, `distance`} |
| SVM | $C \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3\}$<br>$\gamma \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3\}$ |
| Random Forest | $n_{estimators} \in \{100, 200, 500\}$<br>`max_depth` $\in \{1, 3, 5, 10, 30\}$<br>`max_features` $\in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ |
| XGBoost | `max_depth` $\in \{1, 3, 5, 10\}$<br>`reg_alpha`, `reg_lambda` $\in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2, 10^3\}$<br>`subsample`, `colsample_bytree` $\in \{0.1, 0.3, 0.5, 0.7, 0.9\}$<br>early stopping on validation set |

Table 1: Models and hyperparameter grids

## Uncertainty estimation

I report two numbers for each model on the training portion with TimeSeriesSplit(TSCV): the mean F1 score across folds and its standard deviation. The standard deviation of the cross-validation scores measures how sensitive each model is to the exact time window used for validation, i.e. the uncertainty due to the temporal splitting of the data.

Final test performance is reported as single F1 scores for each model. For the best models, variability across folds is still big. It makes sense that for time-sereisCV, the model doesn't learn much in previous folds due to limited amount of data, and in the end the model does learn a lot from more data later.

# Results

## Baseline performance

As a baseline, I use a **stratified random classifier** that samples labels according to the empirical class proportions (78.9% empty, 21.1% occupied). It achieves a mean F1 of $0.20 \pm 0.10$ across TimeSeriesSplit folds and a test F1 of 0.24.

## Cross-validation

Figure 6 summarizes the cross-validation F1-scores. All real models exceed the baseline by a wide margin. XGBoost attains the highest mean score ($0.745 \pm 0.37$), with L1 and L2 logistic regression close behind (0.744 and 0.734). Elastic Net performs similarly (0.71), while KNN and Random Forest reach moderate levels (0.58 and 0.50). The SVM performs poorly.

Relative to baseline ($0.19 \pm 0.10$), XGBoost and L1 logistic regression are roughly $\frac{0.74-0.19}{0.10} \approx 5.5$ standard deviations above it, indicating a substantial improvement over random guessing.

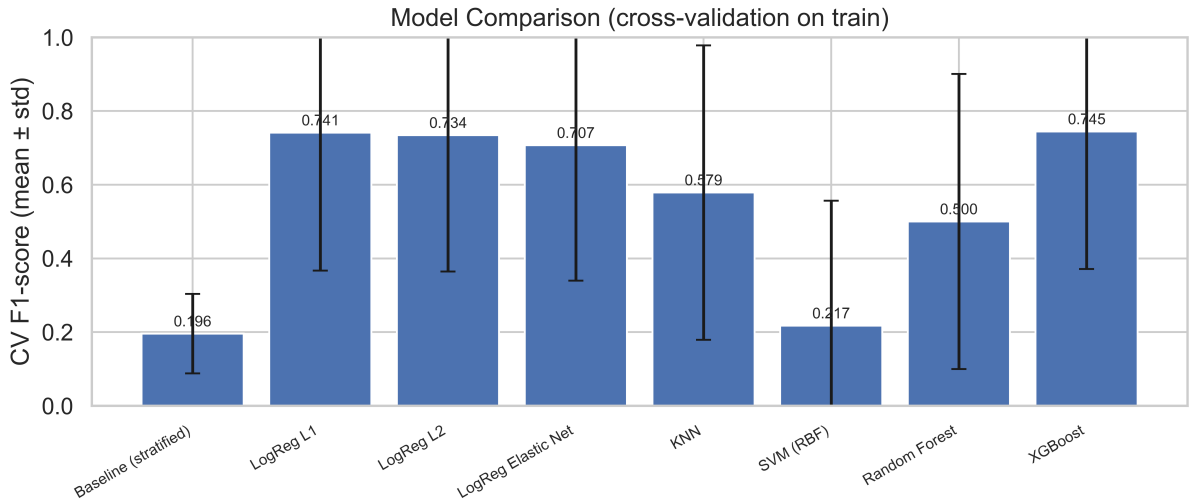


Figure 6: Cross-validation F1-scores (mean ± std) across TimeSeriesSplit folds

## Test performance

Figure 7 shows the test F1-scores using the best hyperparameters for each model. All non-trivial models achieve very high F1 on the held-out test set. L1, L2, and Elastic-Net logistic regression reach test F1 of 0.991, and XGBoost achieves the highest score of 0.992. KNN and Random Forest are slightly worse (0.968 and 0.955), and the RBF SVM is clearly behind (0.899). The stratified baseline remains at 0.248.

Specifically, the results are in Table 2 as following.

Based on this, I use **XGBoost** as the final model: it is the most predictive and remains robust under time-series cross-validation. At the same time, the linear models perform

| Model | CV mean F1 | CV std F1 | Test F1 |
|---|---|---|---|
| Baseline (stratified) | 0.196 | 0.108 | 0.241 |
| Logistic Regression (L1) | 0.741 | 0.374 | 0.991 |
| Logistic Regression (L2) | 0.734 | 0.370 | 0.991 |
| Elastic Net Logistic Regression | 0.707 | 0.368 | 0.991 |
| KNN | 0.579 | 0.400 | 0.968 |
| SVM (RBF) | 0.217 | 0.339 | 0.899 |
| Random Forest | 0.500 | 0.400 | 0.955 |
| XGBoost | 0.745 | 0.374 | 0.992 |

Table 2: Cross-validation and test F1-scores for all models.

almost as well, with only a tiny loss in F1, while training much faster and being easier to interpret through their coefficients. This close performance match between a simple linear model and a complex ensemble also confirms that the underlying decision boundary is not highly non-linear for this task.

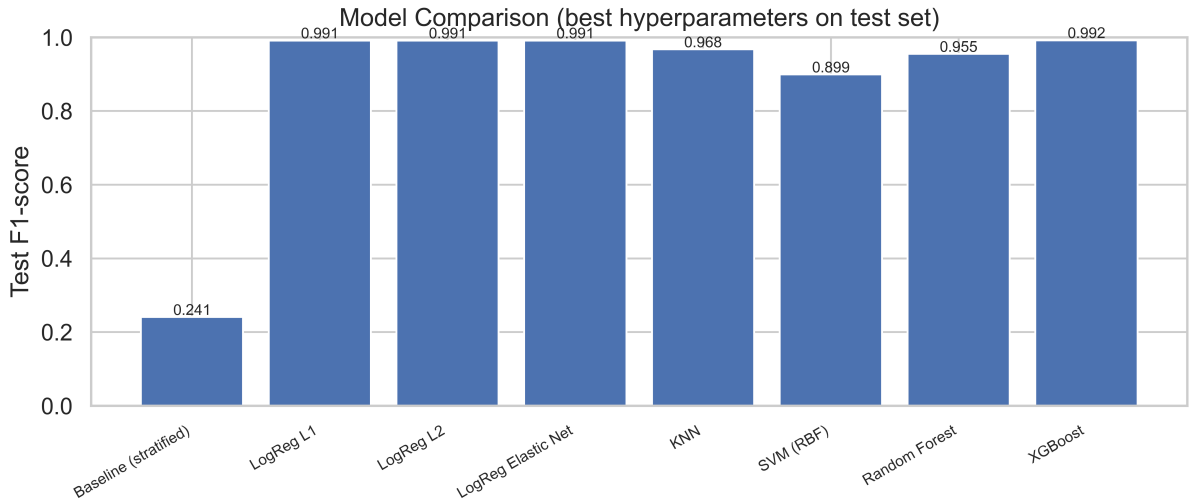


Figure 7: Test F1-scores for the baseline and all models with tuned hyperparameters

## Global feature importance (XGBoost)

I compute three global importance measures for the final XGBoost model: gain-based importance, permutation importance, and SHAP-based importance.

The gain-based ranking (Figure 8) is highly concentrated: **Light** and **Light_lag30** dominate, followed by $CO_2$, $CO_2$_lag30, and Temperature.

Permutation importance on the test set (Figure 9) yields a similar ordering. Shuffling Light or Light_lag30 produces the largest F1 drops, while most other features have small effects, partly due to redundancy with $CO_2$ and its lag.

Aggregated SHAP values (Figures 10–11) again highlight Light and Light_lag30 as the strongest drivers. Low values of Light or $CO_2$ push predictions toward "empty," while high values push toward "occupied." Temporal features have only modest influence, and Humidity contributes almost nothing.

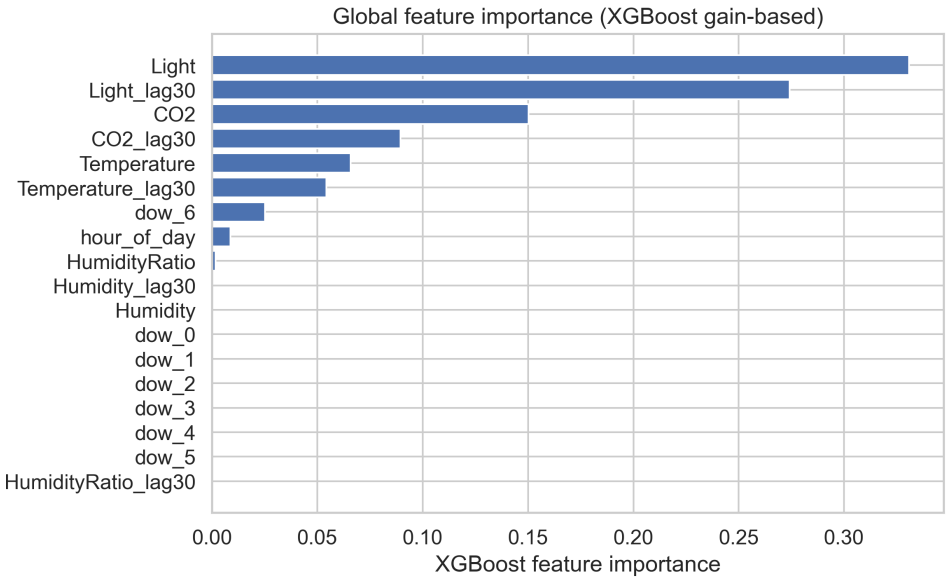


Figure 8: XGBoost gain-based feature importance

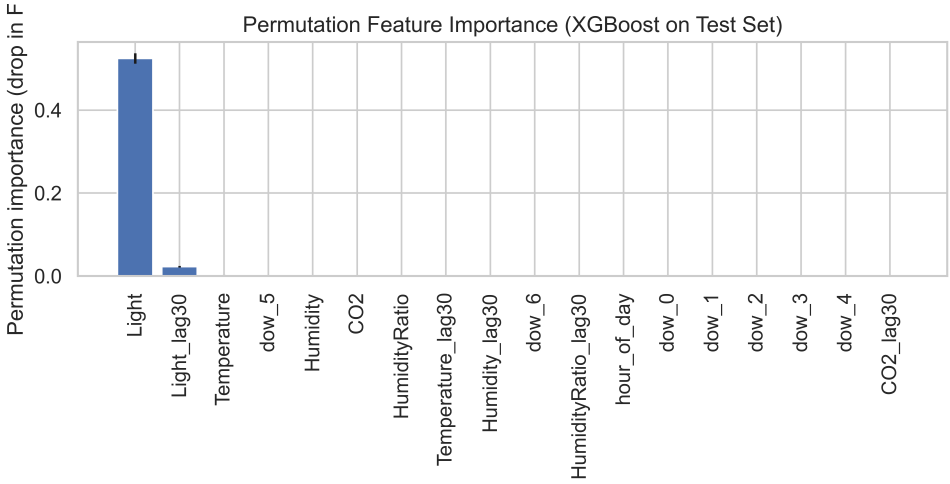


Figure 9: Permutation importance

Across all methods, the conclusion is consistent: current and recent **Light** and **$CO_2$** dominate the model, Temperature plays a secondary role, and Humidity and weekday indicators contribute minimally. This matches expected sensor behavior in a single-room occupancy setting.

## Local interpretability with SHAP

To examine individual XGBoost decisions, I inspect SHAP values on selected test samples. SHAP decomposes each prediction into feature-level contributions, showing which variables push the output toward "occupied" or "empty."

Figure 12 shows a correctly predicted occupied case: high Light, high Light_lag30, and elevated $CO_2$ all contribute positively, which aligns with the expected sensor pattern of

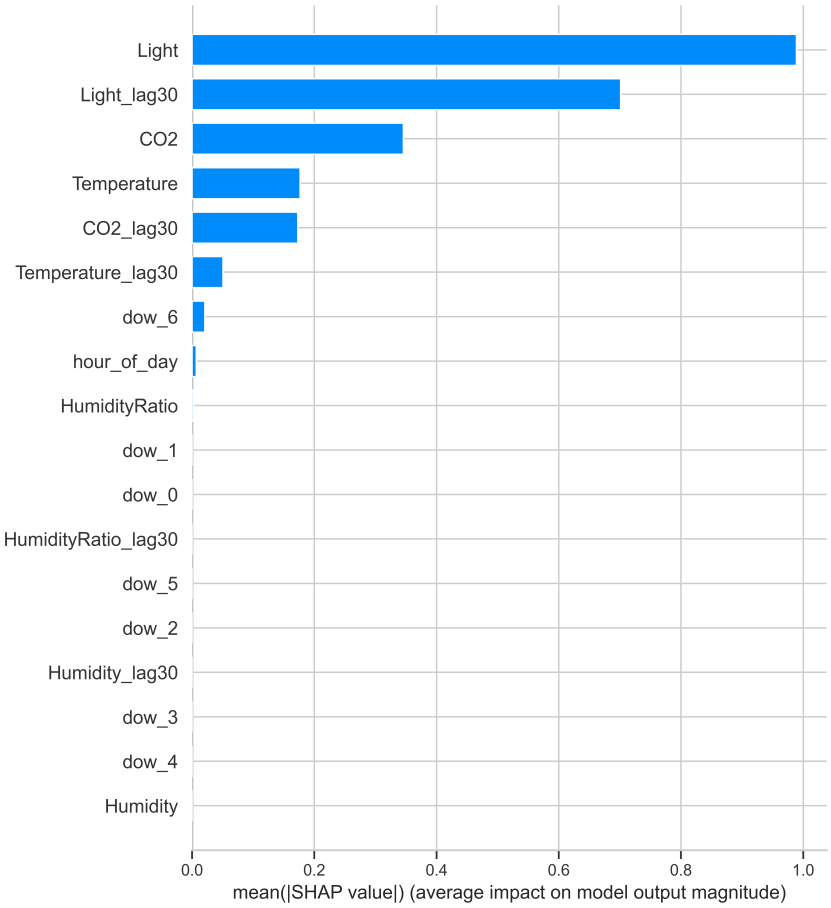


Figure 10: Mean absolute SHAP values (global importance)

an actively used room.

For a correctly predicted empty example (Figure 13), very low Light and low $CO_2$ dominate the negative contributions, pushing the model toward the "empty" class. This again matches the underlying physical intuition.

The error cases are more informative. In a typical false positive (Figure 14), Light is high while $CO_2$ and its lag remain moderate. SHAP shows that the strong positive contribution from Light overrides the weaker $CO_2$ signal, leading the model to predict "occupied" even though the room is empty—an ambiguity that also occurs in practice when lights are left on.

In a false negative example (Figure 15), $CO_2$ and its lag are high, but current Light and Light_lag30 fall near empty-room levels. The negative light contributions outweigh the positive $CO_2$ signal, so the model predicts "empty." This reflects a genuinely ambiguous pattern: an occupied room with lights off.

Across these examples, Light, Light_lag30, and $CO_2$ (and its lag) remain the dominant features, with Temperature playing a smaller role and Humidity contributing little. The remaining errors mostly occur when Light and $CO_2$ disagree, highlighting an inherent limitation of using only these sensors for occupancy detection.

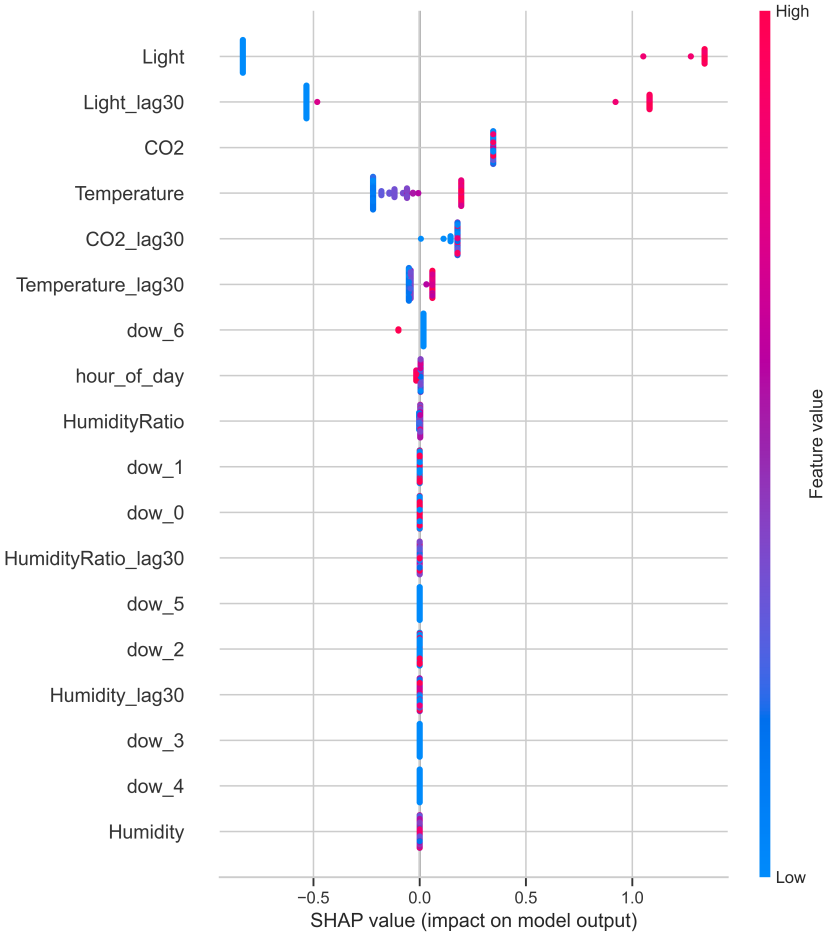


Figure 11: SHAP summary plot for XGBoost

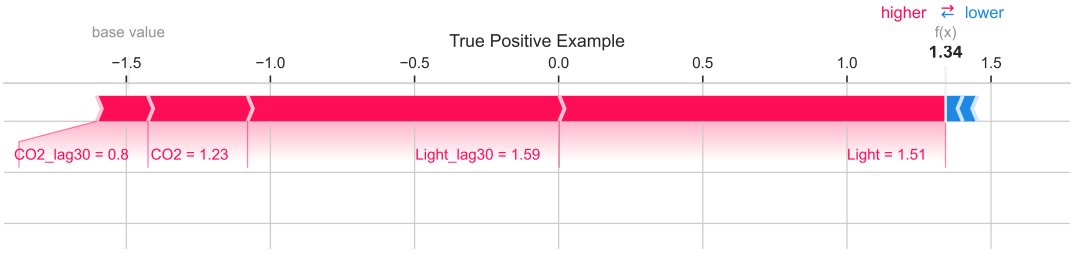


Figure 12: SHAP force plot for a true positive (occupied and predicted occupied)

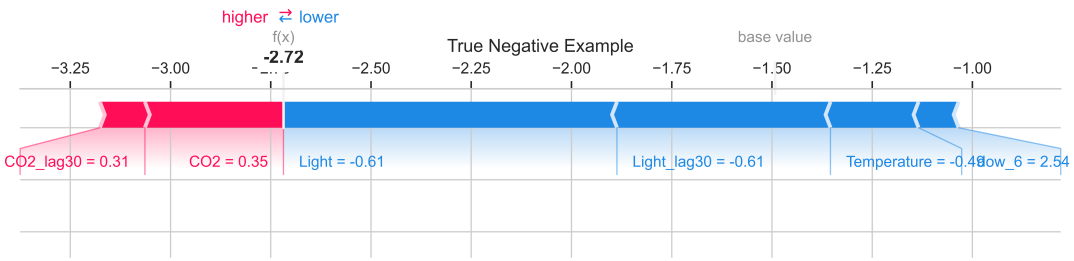


Figure 13: SHAP force plot for a true negative (empty and predicted empty)

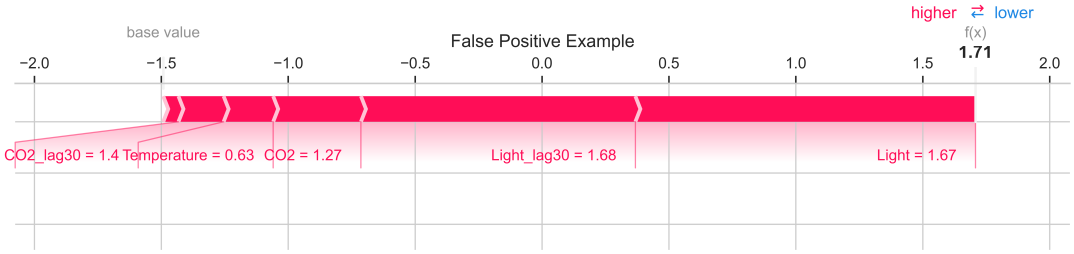


Figure 14: SHAP force plot for a false positive (predicted occupied but actually empty)

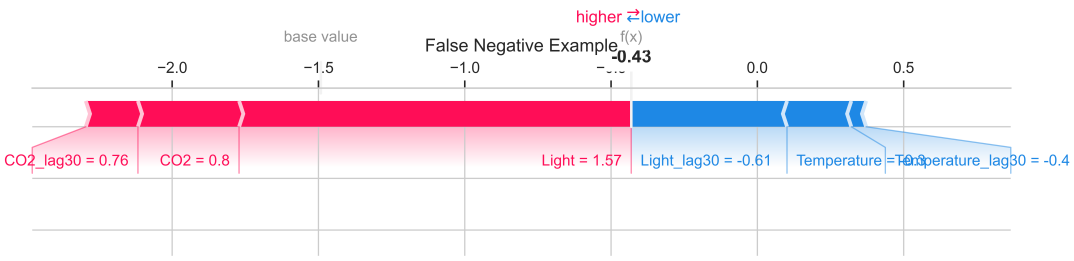


Figure 15: SHAP force plot for a false negative (predicted empty but actually occupied)

# Outlook

The model relies mainly on Light and $CO_2$, and ambiguous situations (e.g. lights off but high $CO_2$) remain difficult to resolve. Additional features could help: longer lag histories, rolling statistics may better capture transient states where the current model struggles.

Further improvements could come from modelling the temporal structure more directly. Approaches such as time-conditioned ensembles, separate models for day vs. night periods, or sequence models (e.g. 1D-CNNs or lightweight-RNNs) could reduce remaining errors by adapting to the different regimes.

Interpretability could also be strengthened. Monotonicity constraints in XGBoost would ensure that physically meaningful relationships (e.g. higher $CO_2$ should not reduce occupancy probability) are respected.

# Bibliography

[1] UCI Machine Learning Repository. *Occupancy Detection Dataset*. Available at: http://archive.ics.uci.edu/dataset/357/occupancy+detection.

[2] Galib, M. *Occupancy Detection UCI Data (GitHub)*. Available at: https://github.com/galib96/occupancy-detection-uci-data.

[3] Turksoy Omer. *HVAC Occupancy Detection with ML and DL Methods*. Kaggle Notebook. Available at: https://www.kaggle.com/code/turksoyomer/hvac-occupancy-detection-with-ml-and-dl-methods.

[4] Pandey, P. *Analyzing Machine Learning Models with Yellowbrick*. Kaggle Notebook. Available at: https://www.kaggle.com/code/parulpandey/analysing-machine-learning-models-with-yellowbrick.